Rozproszony sklep internetowy z oddziałami – analiza i projekt

Przedmiot: Rozproszone bazy danych – projekt

1. Opis problemu i architektury systemu

1.1. Jaki problem rozwiązuje system

System jest prostym sklepem internetowym z dwoma oddziałami: **Katowice** i **Kraków**. Umożliwia prowadzenie wspólnego katalogu produktów, śledzenie stanów magazynowych w każdym oddziale oraz składanie zamówień realizowanych przez wybrany oddział. Celem projektu jest pokazanie, jak dane mogą być rozmieszczone pomiędzy niezależnymi węzłami bazy danych i jak system zachowuje się przy takim podziale.

Problem sprowadza się do pogodzenia dwóch wymagań: produkt powinien być widoczny w obu oddziałach (katalog jest wspólny), a zmiana liczby sztuk lub złożenie zamówienia powinny dotyczyć tylko wybranego oddziału.

1.2. Użytkownicy aplikacji

- **Klient** – przegląda katalog produktów, sprawdza dostępność w oddziałach i składa zamówienie.
- **Pracownik oddziału** – aktualizuje stany magazynowe, przegląda zamówienia swojego oddziału, dodaje nowe produkty do katalogu i wycofuje produkty ze sprzedaży.

Role mają charakter logiczny: aplikacja nie ma logowania ani kontroli uprawnień, więc każda operacja jest dostępna dla każdego, kto ma dostęp do API.

1.3. Przechowywane dane

- katalog produktów (SKU, nazwa, opis, cena, status aktywności),
- stany magazynowe produktów w poszczególnych oddziałach,
- zamówienia (imię i nazwisko klienta, status, wartość) wraz z pozycjami zamówienia (produkt, liczba sztuk, cena jednostkowa).

1.4. Uzasadnienie rozproszonej bazy danych

Każdy oddział najczęściej pracuje na własnych stanach magazynowych i własnych zamówieniach, więc przechowywanie ich lokalnie skraca czas dostępu i pozwala oddziałom działać niezależnie. Awaria jednego węzła nie wyłącza lokalnych operacji drugiego oddziału (zamówień i stanów magazynowych). Jednocześnie katalog produktów i ceny muszą być jednakowe w całej sieci sklepów, dlatego są utrzymywane w obu węzłach. Takie połączenie podziału i powielania danych jest typowym przypadkiem użycia rozproszonej bazy danych.

1.5. Elementy systemu działające w różnych lokalizacjach

- baza danych oddziału Katowice (węzeł 1),
- baza danych oddziału Kraków (węzeł 2),
- warstwa aplikacji (API), która komunikuje się z obydwoma węzłami i potrafi łączyć dane z różnych węzłów w jednej odpowiedzi.

1.6. Architektura systemu

System składa się z trzech głównych usług uruchamianych w Docker Compose:

Usługa	Rola
`api`	Serwer główny i koordynator zapisu katalogu (Python, Starlette, Graphene): udostępnia endpoint GraphQL pod adresem `/graphql`, endpoint `/health` oraz raport testów pod adresem `/reports/tests`.
`postgres_katowice`	Instancja PostgreSQL 16 oddziału Katowice (baza `sklep_katowice`), z włączonymi transakcjami przygotowanymi (`max_prepared_transactions=10`).
`postgres_krakow`	Instancja PostgreSQL 16 oddziału Kraków (baza `sklep_krakow`), z włączonymi transakcjami przygotowanymi (`max_prepared_transactions=10`).
`seed_database`	Usługa jednorazowa, idempotentnie zasilająca oba węzły danymi demonstracyjnymi (dodaje tylko brakujące rekordy).

Usługa	Rola
`test_runner`	Opcjonalny kontener (profil `tests`) uruchamiający zestaw testów bez scenariusza awarii.

Dostęp do baz zrealizowano przez mapowanie obiektowo-relacyjne (PonyORM). Aplikacja przechowuje po jednym połączeniu do każdego węzła i wybiera węzeł na podstawie parametru oddziału lub wykonuje operację na obu węzłach (np. dodanie produktu do katalogu). Dla wygody pracy lokalnej możliwe jest również uruchomienie na dwóch bazach SQLite.

Technologie: Python 3.11, Starlette, Uvicorn, Graphene (GraphQL), PonyORM, PostgreSQL 16, Docker Compose.

1.7. Rola API i zachowanie przy awarii

Serwer główny to usługa `api`. Jest wspólnym punktem wejścia dla klientów i koordynatorem dostępu do danych, a nie centralną bazą: nie przechowuje katalogu, stanów magazynowych ani zamówień. Przyjmuje żądania GraphQL, wybiera węzeł wskazany parametrem `node` dla operacji lokalnych i koordynuje zapis katalogu do obu replik. Dane biznesowe są trwale przechowywane w niezależnych bazach `postgres_katowice` i `postgres_krakow`.

Sytuacja awaryjna	Zachowanie systemu
Niedostępny Kraków	Działają odczyt katalogu, `inventory(node: "KATOWICE")` i zamówienia w Katowicach. Operacje dotyczące Krakowa, odczyty bez parametru `node` i zmiany katalogu zwracają błąd.
Niedostępne Katowice	API działa, ale `products` jest niedostępne, bo katalog jest czytany z Katowic. Działają `inventory(node: "KRAKOW")` i zamówienia w Krakowie. Odczyty bez parametru `node` i zmiany katalogu zwracają błąd.
Niedostępny dowolny węzeł podczas zmiany katalogu	`createProduct` i `deleteProduct` są odrzucane; zmiana nie jest zapisywana tylko w dostępnym węźle ani kolejkowana.

Sytuacja awaryjna	Zachowanie systemu
Niedostępne `api`	Klient nie ma dostępu do GraphQL, ale dane w obu bazach pozostają zachowane. Po ponownym uruchomieniu API dostęp wraca, a przerwane transakcje katalogu są odzyskiwane (sekcja 6.4).

2. Wymagania

2.1. Wymagania funkcjonalne

Nr	Wymaganie
F1	Przeglądanie katalogu aktywnych produktów (SKU, nazwa, opis, cena).
F2	Sprawdzanie stanów magazynowych w wybranym oddziale lub we wszystkich oddziałach jednocześnie.
F3	Dodawanie nowego produktu do katalogu w transakcji dwufazowej PostgreSQL (2PC): produkt wraz z zerowym stanem magazynowym jest zapisywany w obu węzłach albo w żadnym; przy niedostępnym węźle system zwraca błąd i nie zmienia żadnego katalogu, a awaria po podjęciu decyzji commit jest dokańczana automatycznie po odzyskaniu węzła.
F4	Wycofanie produktu ze sprzedaży (oznaczenie jako nieaktywny w obu węzłach) w transakcji 2PC; produkt znika z katalogu i nie można go zamówić, ale nadal jest widoczny w stanach magazynowych.
F5	Aktualizacja stanu magazynowego produktu w wybranym oddziale.

Nr	Wymaganie
F6	Składanie zamówienia w wybranym oddziale z wieloma pozycjami; system sprawdza, czy ilość jest dodatnia, produkt istnieje i jest aktywny oraz czy oddział ma wystarczający stan, a następnie wylicza wartość zamówienia i zmniejsza stan magazynowy.
F7	Przeglądanie zamówień wybranego oddziału lub wszystkich oddziałów wraz z pozycjami.
F8	Zasilenie obu baz danymi demonstracyjnymi (seed); powtórne uruchomienie dodaje tylko brakujące rekordy i nie nadpisuje stanów magazynowych ani flag aktywności.
F9	Zestaw testów generuje raport w formatach HTML, JSON i CSV; raport HTML jest dostępny pod adresem `/reports/tests`.

2.2. Wymagania niefunkcjonalne

Nr	Wymaganie
N1	**Czas odpowiedzi** – czas odczytu katalogu zależy od liczby zwracanych produktów: w testach wyniósł około 521–567 ms dla 131 produktów, 4 346–4 461 ms dla 1 031 produktów i 42 166–45 777 ms dla 10 031 produktów (sekcja 10.1). Średni czas odczytu w teście obciążenia wyniósł około 139–159 ms (sekcja 10.2), a zmiana katalogu w transakcji 2PC około 105–133 ms (sekcja 10.6).
N2	**Dostępność** – awaria jednego węzła nie uniemożliwia pracy z lokalnymi danymi drugiego oddziału (zamówienia i stany magazynowe), natomiast zapytania obejmujące oba węzły oraz zmiany katalogu zwracają błąd, a przy awarii Katowic niedostępny jest też odczyt katalogu (sekcja 10.4).

Nr	Wymaganie
N3	**Odporność na awarie** – po ponownym uruchomieniu węzła system wraca do poprawnego działania bez utraty danych (dane są na trwałych wolumenach); transakcje katalogu przerwane awarią są wycofywane albo dokańczane automatycznie (sekcje 6.4 i 10.5).
N4	**Spójność danych** – zmiany katalogu są zapisywane w transakcji dwufazowej PostgreSQL (2PC): obie repliki są przygotowywane, koordynator zapisuje trwałą decyzję commit w Katowicach i dopiero wtedy zatwierdza obie. Awaria przed decyzją wycofuje zapis, a po decyzji zapis jest dokańczany przy kolejnej zmianie katalogu lub restarcie API; do tego czasu zmiana może być widoczna tylko w jednym węźle. Zmiany wykonane poza aplikacją nie są synchronizowane; stany magazynowe i zamówienia są spójne w obrębie swojego węzła (transakcje lokalne).
N5	**Bezpieczeństwo dostępu** – dane dostępowe do baz aplikacja pobiera ze zmiennych środowiskowych, a użytkownik korzysta z danych przez API GraphQL. System nie ma logowania ani autoryzacji, a porty baz (5433 i 5434) są w konfiguracji demonstracyjnej wystawione na hosta, więc zabezpieczenie dostępu jest ograniczone.
N6	**Skalowalność** – czas odpowiedzi rośnie liniowo z liczbą danych i operacji (sekcja 10); dodanie kolejnego oddziału wymaga zmian w kodzie, bo węzły Katowice i Kraków są zdefiniowane na stałe w pliku models.py.
N7	**Przenośność** – całość środowiska jest skonfigurowana do uruchomienia jednym poleceniem `docker compose up --build` albo lokalnie na dwóch plikach SQLite.

Nr	Wymaganie
N8	**Obserwowalność** – dostępne są endpoint `/health`, logi Docker Compose i raport testów w HTML (`/reports/tests`).
N9	**Powtarzalność** – seed jest idempotentny (dodaje brakujące rekordy, nie nadpisuje stanów magazynowych ani flag `active`), a dane testowe `PERF-*` i `CONS-*` są usuwane po teście.

3. Diagram przypadków użycia UML

3.1. Aktorzy i przypadki użycia

Aktor	Przypadki użycia
Klient	Przeglądanie katalogu produktów (UC1); Sprawdzenie dostępności produktu w oddziałach (UC2); Złożenie zamówienia (UC3)
Pracownik oddziału	Sprawdzenie dostępności produktu w oddziałach (UC2); Aktualizacja stanu magazynowego (UC4); Przeglądanie zamówień oddziału (UC5); Dodanie produktu do katalogu (UC6); Wycofanie produktu ze sprzedaży (UC7)

Zależności między przypadkami użycia (notacja UML): UC3 (Złożenie zamówienia) «include» UC8 (Sprawdzenie aktywności i stanu produktu); UC6 (Dodanie produktu do katalogu) i UC7 (Wycofanie produktu ze sprzedaży) «include» UC9 (Zapis zmiany katalogu w obu węzłach w transakcji 2PC). Przy niedostępnym węźle UC9 kończy się błędem bez zmian w katalogach.

Klient i pracownik oddziału współdzielą sprawdzanie dostępności

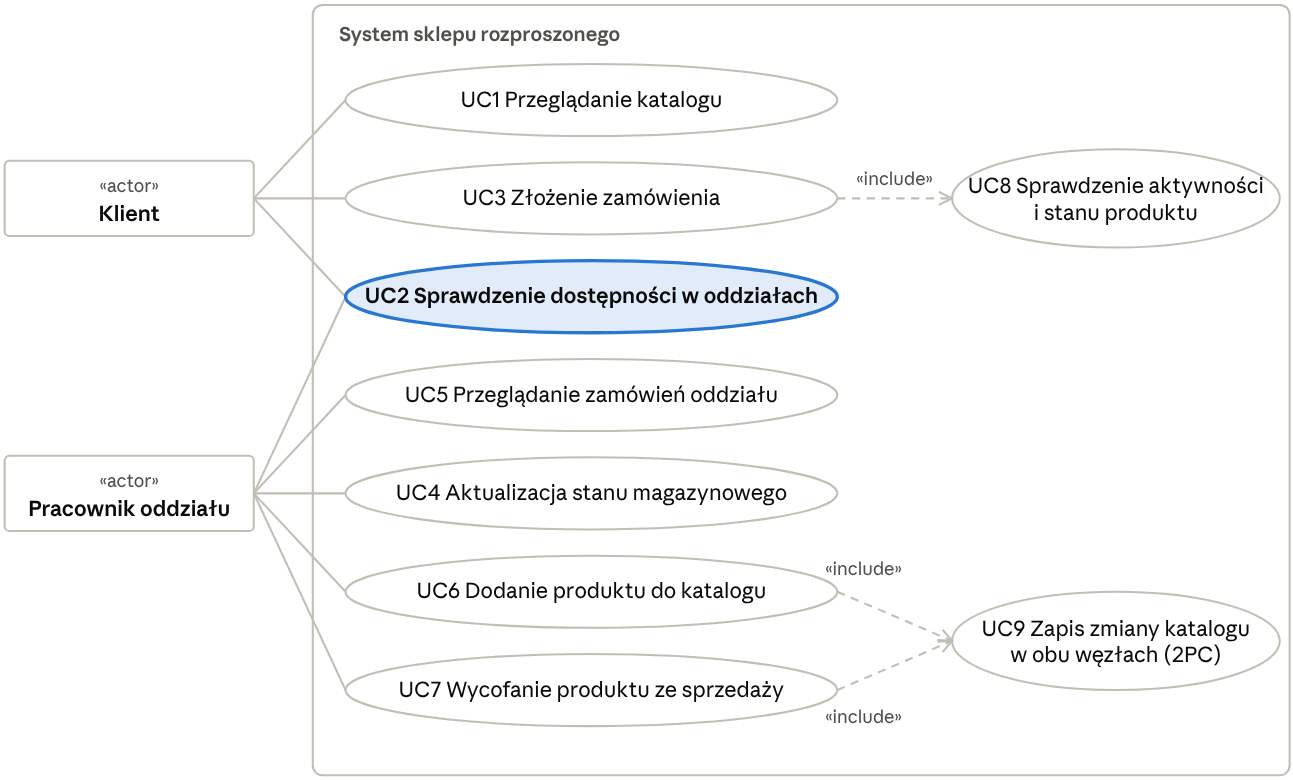


diagram przypadków użycia UML · 2 aktorów, 9 przypadków, 3 zależności «include»

Klient i pracownik oddziału korzystają wspólnie z UC2, a pracownik oddziału dodatkowo zarządza produktami w obu węzłach (UC6, UC7).

4. Scenariusze użycia

Scenariusz 1: Zakup produktu

Aktor: Klient

Warunek początkowy: Produkt jest aktywny w katalogu, a wybrany oddział (np. Katowice) ma go na stanie w wystarczającej liczbie.

Przebieg:

1. Klient przegląda katalog i wybiera produkty oraz oddział realizujący zamówienie.
2. Klient podaje swoje dane i listę pozycji (SKU oraz liczba sztuk).
3. System odnajduje produkty w węźle wybranego oddziału i sprawdza, czy są aktywne oraz czy oddział ma ich wystarczająco dużo.

4. System zapisuje zamówienie ze statusem NEW, dodaje jego pozycje z aktualną ceną jednostkową i wylicza wartość zamówienia.
5. System zmniejsza stan magazynowy w węźle wybranego oddziału.

Rezultat: Zamówienie zostaje zapisane wyłącznie w węźle wybranego oddziału, a stan magazynowy tego oddziału jest pomniejszony. Jeśli produktu brakuje, jest nieaktywny, ilość nie jest dodatnia lub jest go za mało, system zwraca błąd, a zamówienie nie jest zapisane.

Scenariusz 2: Sprawdzenie dostępności produktu w oddziałach

Aktor: Klient (lub pracownik oddziału)

Warunek początkowy: Oba węzły są dostępne, a w katalogu istnieją produkty.

Przebieg:

1. Użytkownik wysyła zapytanie o stany magazynowe bez wskazywania konkretnego oddziału.
2. System odpytuje węzeł Katowice oraz węzeł Kraków.
3. System łączy wyniki obu węzłów w jedną listę z informacją o oddziale, produkcie i liczbie sztuk.

Rezultat: Użytkownik widzi, w którym oddziale i w jakiej liczbie produkt jest dostępny. Jest to operacja wykorzystująca dane z więcej niż jednego węzła.

Scenariusz 3: Dodanie nowego produktu do katalogu

Aktor: Pracownik oddziału

Warunek początkowy: Produkt o danym SKU nie istnieje jeszcze w katalogu.

Przebieg:

1. Administrator podaje SKU, nazwę, opis i cenę nowego produktu.
2. System w jednej transakcji dwufazowej (2PC) sprawdza, czy SKU jeszcze nie istnieje, dodaje produkt i zerowy stan magazynowy w węźle Katowice i w węźle Kraków, przygotowuje obie transakcje, zapisuje decyzję commit i zatwierdza je w obu węzłach.
3. System tworzy w każdym węźle rekord stanu magazynowego z zerową liczbą sztuk.
4. Następnie ten sam pracownik ustawia początkowy stan magazynowy w swoim oddziale.

Rezultat: Nowy produkt jest widoczny w katalogu obu oddziałów, a stany magazynowe są niezależne w każdym z nich. Jeśli któryś węzeł jest niedostępny, zmiana jest wycofana, a gdy awaria nastąpi po podjęciu decyzji commit, zostaje dokończona po odzyskaniu węzła.

5. Diagram bazy danych

5.1. Encje i relacje

Każdy węzeł ma identyczny schemat tabel. Różnią się danymi: tabele `products` mają w obu węzłach tę samą treść (katalog), a tabele `inventory`, `orders` i `order_items` zawierają dane lokalne dla oddziału.

Katalog produktów jest wspólny, pozostałe tabele są lokalne dla oddziału

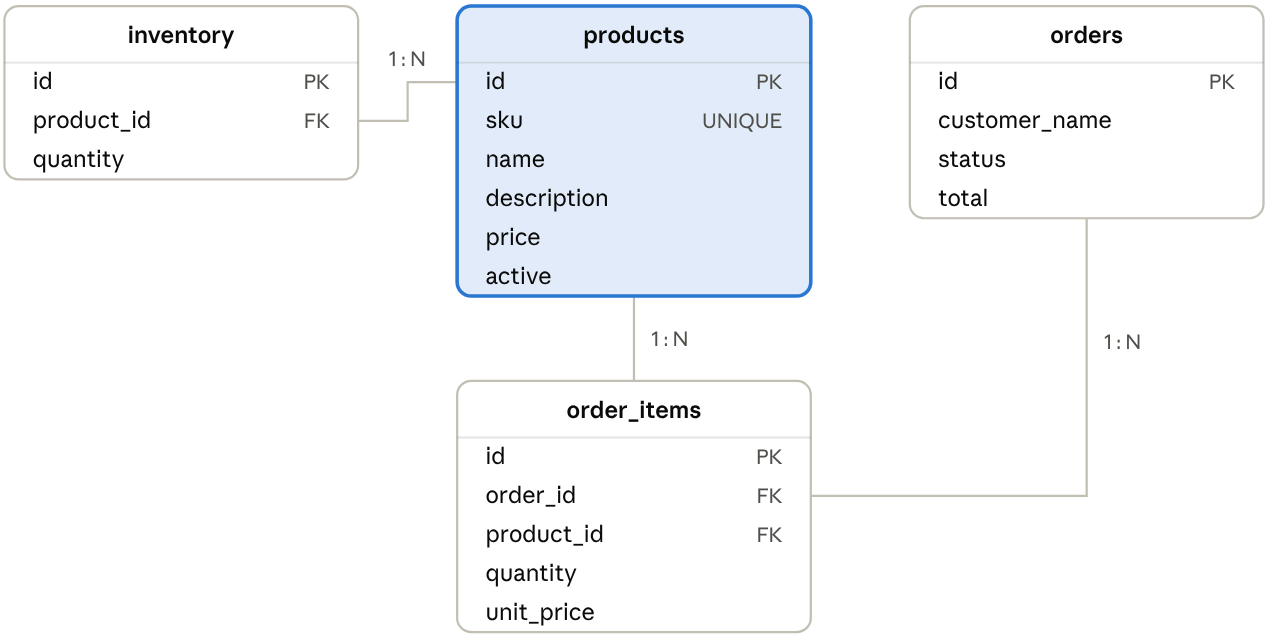


diagram ERD · 4 tabele, 3 relacje

Tabela products ma tę samą treść w obu węzłach, a inventory, orders i order_items zawierają dane lokalne oddziału.

products – współdzielony katalog produktów.

Pole	Typ	Klucz / ograniczenie
id	int	PK

Pole	Typ	Klucz / ograniczenie
sku	varchar(50)	UNIQUE, NOT NULL
name	varchar(200)	NOT NULL
description	varchar(1000)	
price	float	NOT NULL
active	boolean	NOT NULL, domyślnie true

inventory – liczba sztuk produktu dostępna w danym węźle (oddziale).

Pole	Typ	Klucz / ograniczenie
id	int	PK
product_id	int	FK → products.id
quantity	int	NOT NULL, domyślnie 0

orders – zamówienia klientów zapisane lokalnie w wybranym oddziale.

Pole	Typ	Klucz / ograniczenie
id	int	PK
customer_name	varchar(200)	NOT NULL
status	varchar(30)	NOT NULL, domyślnie NEW
total	float	NOT NULL, domyślnie 0

order_items – pozycje zamówienia (produkt i liczba sztuk).

Pole	Typ	Klucz / ograniczenie
id	int	PK
order_id	int	FK → orders.id
product_id	int	FK → products.id
quantity	int	NOT NULL

Pole	Typ	Klucz / ograniczenie
unit_price	float	NOT NULL

5.2. Relacje między tabelami

- products 1 : N inventory – produkt ma stan magazynowy w danym węźle,
- products 1 : N order_items – produkt może występować w wielu pozycjach zamówień,
- orders 1 : N order_items – zamówienie składa się z jednej lub wielu pozycji.

5.3. Przeznaczenie tabel

- **products** – katalog produktów wspólny dla wszystkich oddziałów.
- **inventory** – dostępność produktów w konkretnym oddziale.
- **orders** – informacje o zakupach klientów w oddziale.
- **order_items** – szczegóły zamówienia: co i w jakiej liczbie zostało zamówione oraz po jakiej cenie.

Poza tabelami biznesowymi w węźle Katowice działa techniczna tabela `catalog_transaction_decisions` (pola `gid`, `decision`, `created_at`). Koordynator transakcji 2PC zapisuje w niej trwałą decyzję commit, zanim zatwierdzi zmianę w obu węzłach (sekcja 6.4).

6. Sposób rozproszenia danych

Dane są rozmieszczone w dwóch niezależnych instancjach PostgreSQL: **węzeł Katowice** (baza `sklep_katowice`) i **węzeł Kraków** (baza `sklep_krakow`). Zastosowano połączenie powielania katalogu i fragmentacji poziomej danych oddziału.

Tabela	Sposób rozmieszczenia	Katowice	Kraków
products	powielenie (pełna kopia w każdym węźle)	pełny katalog	pełny katalog
inventory	dane lokalne oddziału	stany magazynu Katowic	stany magazynu Krakowa
orders	fragmentacja pozioma	zamówienia złożone w Katowicach	zamówienia złożone w Krakowie

Tabela	Sposób rozmieszczenia	Katowice	Kraków
order_items	fragmentacja pozioma (wraz z zamówieniem)	pozycje zamówień Katowic	pozycje zamówień Krakowa

6.1. Jak to działa

- **Katalog produktów** jest powielany mechanizmem aplikacyjnym, a nie replikacją silnika bazy danych. Mutacje `createProduct` i `deleteProduct` zapisują zmianę w obu węzłach w jednej transakcji dwufazowej PostgreSQL (2PC, sekcja 6.4). Odczyt katalogu (`products`) korzysta z węzła Katowice.
- **Stany magazynowe i zamówienia** są zapisywane wyłącznie w węźle wskazanym parametrem `node` . Złożenie zamówienia w jednym oddziale nie zmienia stanu magazynu w drugim.
- **Zapytania wielowęzłowe:** `inventory` i `orders` wywołane bez parametru `node` odpytują oba węzły i łączą wyniki w jedną listę, w której każdy rekord ma pole `node` .
- **Identyfikatory:** każdy węzeł ma własne sekwencje, więc `id` nie jest globalnie unikalne (zamówienie nr 1 istnieje i w Katowicach, i w Krakowie). Rekordy odróżnia para (`node` , `id`), a produkty globalny klucz biznesowy `sku` .

6.2. Uzasadnienie

Stany magazynowe i zamówienia są najczęściej używane właśnie w oddziale, który je tworzy, więc trzymanie ich lokalnie skraca dostęp i izoluje oddziały od awarii siebie nawzajem. Katalog produktów jest potrzebny w każdym oddziale i zmienia się rzadko, więc jego kopia w każdym węźle nie jest kosztowna.

Typ danych	Strategia	Uzasadnienie
Katalog produktów	powielenie (mechanizm aplikacyjny)	każdy oddział potrzebuje pełnego, jednakowego katalogu, a zmiany są rzadkie
Stany magazynowe	fragmentacja pozioma według oddziału	stan fizyczny dotyczy lokalnego magazynu
Zamówienia	fragmentacja pozioma według oddziału	sprzedaż jest obsługiwana lokalnie

6.3. Ograniczenia tego podejścia

- Transakcja dwufazowa jest blokująca: jeśli węzeł padnie po podjęciu decyzji commit, jego przygotowana transakcja czeka w stanie prepared (zgodnie z działaniem PostgreSQL trzyma wtedy blokady), a kolejne zmiany katalogu kończą się błędem do czasu odzyskania węzła.
- Do czasu dokończenia transakcji zatwierdzona zmiana może być widoczna tylko w jednym węźle (przejściowa niespójność, sekcja 10.5). Odzyskiwanie uruchamia się przy starcie API i przed każdą zmianą katalogu, a nie w tle, więc przy braku nowych zmian i restartu przygotowana transakcja czeka.
- Koordynatorem jest API, a trwała decyzja commit jest zapisywana w Katowicach. Gdy Katowice są niedostępne, zmiany katalogu są odrzucane.
- Zmiana wykonana bezpośrednio w bazie jednego węzła, z pominięciem aplikacji, nie jest przenoszona do drugiego (sekcja 10.3).
- Odczyt katalogu zależy od węzła Katowice.

6.4. Transakcja dwufazowa (2PC) przy zapisie katalogu

Zmiany katalogu (`createProduct` i `deleteProduct`) wykonuje moduł `backend/catalog_transaction.py` z użyciem mechanizmu PostgreSQL `PREPARE TRANSACTION` i `COMMIT PREPARED` . Koordynatorem jest API:

1. **Odzyskiwanie:** koordynator dokańcza albo wycofuje transakcje przerwane we wcześniejszych próbach.
2. **Faza przygotowania:** API otwiera połączenia do obu węzłów, wykonuje zmianę (sprawdzenie SKU, wstawienie produktu i zerowego stanu magazynowego albo ustawienie `active = false`) i na każdym węźle wywołuje `PREPARE TRANSACTION` .
3. **Decyzja:** koordynator trwale zapisuje decyzję `commit` w tabeli `catalog_transaction_decisions` w Katowicach.
4. **Faza zatwierdzenia:** na obu węzłach wykonywane jest `COMMIT PREPARED` .

Błąd w fazie przygotowania lub brak zapisanej decyzji powoduje `ROLLBACK PREPARED` na przygotowanych węzłach. Odzyskiwanie jest idempotentne i uruchamiane przy starcie API oraz przed każdą zmianą katalogu. Mechanizm działa tylko dla PostgreSQL (`DB_PROVIDER=postgres`).

Moment awarii	Skutek	Zakończenie
Węzeł niedostępny przed zapisem	Błąd, brak zmian i brak transakcji przygotowanych	Nie wymaga naprawy
Awaria API po przygotowaniu obu węzłów, przed decyzją	Transakcje czekają w stanie prepared, produkt jest niewidoczny	Wycofanie przy restarcie API lub przy następnej zmianie katalogu
Awaria węzła po decyzji commit	Pierwszy węzeł zatwierdzony (produkt widoczny), drugi w stanie prepared	Dokończenie commit po odzyskaniu węzła, przy następnej zmianie katalogu lub restarcie API

Wyniki tych scenariuszy sprawdzono testami opisanymi w sekcji 10.5.

7. Implementacja

Kod projektu znajduje się w repozytorium: https://github.com/kmichal661/projekt-rozproszone-bazy-danych

7.1. Struktura

Plik	Rola
`backend/models.py`	Modele (PonyORM) i połączenia z obiema bazami; funkcje `list_products`, `list_inventory`, `list_orders`.
`backend/schema.py`	Schemat GraphQL: zapytania `products`, `inventory`, `orders`.
`backend/mutations.py`	Mutacje `createProduct`, `updateInventory`, `createOrder`, `deleteProduct`.
`backend/seed_database.py`	Jednorazowe, powtarzalne zasilenie obu węzłów danymi przykładowymi.
`backend/app.py`	Aplikacja Starlette: endpoint `/graphql` i `/health`.
`docker-compose.yml`	Dwa kontenery PostgreSQL 16 (porty 5433 i 5434), usługa seed i API (port 8000).

Moduł `backend/catalog_transaction.py` realizuje transakcję dwufazową (2PC) dla zmian katalogu: koordynuje `PREPARE TRANSACTION` i `COMMIT PREPARED` w obu węzłach, zapisuje decyzję commit w tabeli `catalog_transaction_decisions` i odzyskuje przerwane transakcje (sekcja 6.4). Korzystają z niego mutacje `createProduct` i `deleteProduct` z pliku `backend/mutations.py`.

Repozytorium zawiera też katalog `Tests/` z automatycznym zestawem testów. Zapisuje on wyniki w `Tests/results/` (`report.html`, `report.json`, `measurements.csv`), a raport HTML jest dostępny pod adresem `/reports/tests`. Komendy uruchamiania testów podano w sekcji 7.2.

7.2. Uruchomienie

```
docker compose up -d --build
docker compose ps
```

Po starcie interfejs GraphiQL jest dostępny pod adresem `http://localhost:8000/graphql`, a sprawdzenie stanu usługi pod adresem `http://localhost:8000/health`.

Obie instancje PostgreSQL muszą mieć ustawione `max_prepared_transactions` większe od 0 (w `docker-compose.yml` jest to `max_prepared_transactions=10`), bo transakcja dwufazowa korzysta z `PREPARE TRANSACTION`. Transakcje 2PC są używane tylko przy `DB_PROVIDER=postgres`; w trybie SQLite katalog jest zapisywany bez 2PC, z samym sprawdzeniem obu baz przed zapisem.

Reset danych PostgreSQL (usuwa wolumeny):

```
docker compose down -v
docker compose up -d --build
```

Uruchomienie lokalne bez Dockera (dwa pliki SQLite: `data/katowice/katowice.db` i `data/krakow/krakow.db`):

```
pip install -r requirements.txt
python -m backend.seed_database
python backend/app.py
```

Uruchomienie testów (pełny zestaw albo w kontenerze, bez testu awarii):

```
python -m Tests.run_tests
docker compose --profile tests run --rm test_runner
```

7.3. Operacje CRUD

Operacja	Funkcja w API
CREATE	`createProduct` (oba węzły), `createOrder` (wybrany węzeł)
READ	`products`, `inventory`, `orders` (jeden węzeł lub oba)
UPDATE	`updateInventory`; `createOrder` dodatkowo zmniejsza stan magazynowy
DELETE	`deleteProduct` (usunięcie logiczne: `active = false` w obu węzłach)

Operacja wykorzystująca dane z więcej niż jednego węzła: `inventory` bez parametru `node` (sprawdzenie, gdzie produkt jest dostępny).

7.4. Środowisko, w którym wykonano testy

Testy z sekcji 9 i 10 wykonano lokalnie na dwóch niezależnych instancjach PostgreSQL 16 (porty 5433 i 5434, takie same jak w `docker-compose.yml`) i Pythonie 3.11, z danymi, użytkownikiem i nazwami baz z pliku `docker-compose.yml`. Kontenerów Docker w tym środowisku nie uruchamiano. Pony ORM 0.7.17 nie działa na Pythonie 3.12, więc należy zachować wersję 3.11 z pliku `Dockerfile`.

Zestaw `Tests/` z repozytorium (bez testu awarii, który wymaga Dockera) przeszedł w tym środowisku w całości: czas odpowiedzi, obciążenie, spójność i skalowalność. Test awarii wykonano ręcznie, wyłączając bazę Krakowa (sekcja 10.4).

W środowisku testowym ustawiono `max_prepared_transactions=10` na obu instancjach PostgreSQL, tak jak w `docker-compose.yml`. Zestaw `Tests/` przeszedł również po dodaniu transakcji 2PC.

7.5. Usterki znalezione i poprawione

Podczas testów wykryto trzy usterki, które poprawiono w aktualnej wersji kodu. Endpoint `/health` zwracał błąd 500 z powodu brakującego importu `JSONResponse` w `backend/app.py`. Mutacja `createOrder` nie sprawdzała flagi `active`, więc produkt wycofany przez `deleteProduct` można było zamówić. Mutacje `createProduct` i `deleteProduct` przy niedostępnym węźle zostawiały zmianę tylko w Katowicach, a obecnie używają transakcji dwufazowej PostgreSQL (sekcja 6.4).

8. Przykładowe dane

Skrypt `seed_database.py` wprowadza do każdego węzła ten sam katalog 30 produktów i osobne stany magazynowe. W tabeli pokazano 6 produktów nazwanych:

SKU	Nazwa	Cena (zł)	Stan Katowice	Stan Kraków
LAP-001	Laptop biznesowy	3499,99	10	7
MON-001	Monitor 27 cali	1199,00	15	9
KEY-001	Klawiatura mechaniczna	299,00	20	12
MOU-001	Mysz bezprzewodowa	149,99	25	18
HED-001	Słuchawki bezprzewodowe	399,00	8	11

SKU	Nazwa	Cena (zł)	Stan Katowice	Stan Kraków
WEB-001	Kamera internetowa	249,00	14	6

Seed tworzy 240 rekordów: w każdym węźle 30 produktów (6 z tabeli powyżej i 24 wygenerowane o SKU od `SEED-001` do `SEED-024`), 30 stanów magazynowych, 30 zamówień i 30 pozycji zamówień. Ponowne uruchomienie seeda niczego nie dubluje (w sprawdzeniu utworzył 0 nowych rekordów), więc już po starcie liczba rekordów przekracza około 30 wymaganych w wytycznych. Do testów wydajnościowych dane generowano skryptem (sekcja 10.1).

Seed jest idempotentny: dodaje tylko brakujące rekordy, nie przywraca flagi `active = true` dla wcześniej wycofanego produktu i nie zmienia istniejących stanów magazynowych.

9. Zapytania testowe

Zapytania wysyłano do endpointu `/graphql` na danych z sekcji 8. Wyniki poniżej są skrócone tam, gdzie lista byłaby długa.

9.1. Odczyt katalogu

```
{
  products {
    sku
    name
    price
    active
  }
}
```

Cel: pobranie aktywnych produktów z katalogu (odczyt z węzła Katowice).

Wynik: 30 produktów (36 ms), np.

```
{"data": {"products": [
  {"sku": "LAP-001", "name": "Laptop biznesowy", "price": 3499.99, "active":
true},
  {"sku": "MON-001", "name": "Monitor 27 cali", "price": 1199.0, "active":
true},
  ...
  {"sku": "SEED-024", "name": "Produkt demonstracyjny 024", "price": 229.99,
"active": true}
]}}
```


9.2. Dostępność produktów w obu oddziałach (dane z dwóch węzłów)

```
{
  inventory {
    node
    quantity
    product { sku }
  }
}
```

Cel: sprawdzenie, gdzie i w jakiej liczbie produkty są dostępne. Zapytanie bez parametru `node` odpytuje oba węzły i łączy wyniki w jedną listę.

Wynik: 60 rekordów (74 ms), po 30 z węzła KATOWICE i z węzła KRAKOW, np.


```
{"node": "KATOWICE", "quantity": 10, "product": {"sku": "LAP-001"}}
{"node": "KRAKOW", "quantity": 7, "product": {"sku": "LAP-001"}}
```

9.3. Dodanie produktu do katalogu (CREATE, oba węzły)

```
mutation {
  createProduct(product: {
    sku: "SSD-001"
    name: "Dysk SSD 1 TB"
    description: "Dysk NVMe"
    price: 449.0
  }) {
    ok
    message
    product { id sku name price active }
  }
}
```

Cel: dodanie nowego produktu do katalogu w obu węzłach.

Wynik: `ok = true`, produkt zapisany w obu węzłach.


```
{"data": {"createProduct": {"ok": true, "message": "Produkt zapisano atomowo
w obu węzłach (PostgreSQL 2PC)",
  "product": {"id": 31, "sku": "SSD-001", "name": "Dysk SSD 1 TB", "price":
449.0, "active": true}}}}
```


9.4. Zmiana stanu magazynowego (UPDATE)

```
mutation {
  updateInventory(node: "KATOWICE", sku: "SSD-001", quantity: 20) {
    ok
    message
    inventory { node sku quantity }
  }
}
```

```
mutation {
  updateInventory(node: "KRAKOW", sku: "SSD-001", quantity: 5) {
    ok
    message
    inventory { node sku quantity }
  }
}
```

Cel: ustawienie początkowych stanów nowego produktu osobno w każdym oddziale.

Wynik: `ok = true` dla obu zapytań, a stan zmienia się tylko w wskazanym węźle.

```
{"updateInventory": {"ok": true, "message": "Stan magazynowy zaktualizowany",
"inventory": {"node": "KATOWICE", "sku": "SSD-001", "quantity": 20}}}
{"updateInventory": {"ok": true, "message": "Stan magazynowy zaktualizowany",
"inventory": {"node": "KRAKOW", "sku": "SSD-001", "quantity": 5}}}
```

9.5. Złożenie zamówienia (CREATE i UPDATE w jednym węźle)

```
mutation {
  createOrder(
    node: "KRAKOW"
    customerName: "Anna Nowak"
    items: [
      { sku: "LAP-001", quantity: 2 }
      { sku: "MOU-001", quantity: 3 }
    ]
  ) {
    ok
    message
    order { id node total status }
  }
}
```

Cel: zapisanie zamówienia w Krakowie i zmniejszenie stanu magazynowego w Krakowie.

Wynik: `ok = true`, zamówienie o wartości 7449,95 zł.

```
{"data": {"createOrder": {"ok": true, "message": "Zamówienie utworzone",
  "order": {"id": 31, "node": "KRAKOW", "total": 7449.95, "status": "NEW"}}}}
```

Po zamówieniu stany w Krakowie spadły: LAP-001 z 7 do 5, MOU-001 z 18 do 15, a w Katowicach pozostały bez zmian. Zamówienie na 500 sztuk LAP-001 zostało odrzucone błędem `Brak wystarczającej ilości produktu LAP-001` i nie zostawiło w bazie żadnego zamówienia.

9.6. Zamówienia z całego systemu (dane z dwóch węzłów)

```
{
  orders {
    id
    node
    customerName
    status
    total
    items { quantity unitPrice product { sku } }
  }
}
```

Cel: zestawienie zamówień ze wszystkich oddziałów. Drugie zamówienie (4 szt. SSD-001) złożono w Katowicach.

Wynik: 62 zamówienia, po 31 z każdego węzła. Poniżej dwa nowo dodane:

```
{"id": 31, "node": "KATOWICE", "customerName": "Jan Kowalski", "status":
"NEW", "total": 1796.0,
 "items": [{"quantity": 4, "unitPrice": 449.0, "product": {"sku": "SSD-
001"}}]}
{"id": 31, "node": "KRAKOW", "customerName": "Anna Nowak", "status": "NEW",
"total": 7449.95,
 "items": [{"quantity": 3, "unitPrice": 149.99, ...}, {"quantity": 2,
"unitPrice": 3499.99, ...}]}
```

9.7. Zapytania agregujące (COUNT, SUM) w każdym węźle

```
SELECT COUNT(*) AS zamowienia, COALESCE(SUM(total), 0) AS przychod FROM
orders;
SELECT SUM(quantity) AS sztuk_w_magazynie FROM inventory;
```

Cel: liczba zamówień, przychód i liczba sztuk w magazynie. Zapytanie SQL wykonano osobno na każdej bazie.

Wynik:

Węzeł	Zamówienia	Przychód (zł)	Sztuk w magazynie
Katowice	31	11 041,74	888
Kraków	31	16 695,69	723

Węzeł	Zamówienia	Przychód (zł)	Sztuk w magazynie
Razem	62	27 737,43	1 611

Sumę „Razem” wyliczono ręcznie z wyników obu węzłów, ponieważ aplikacja nie ma gotowego zapytania zbiorczego.

9.8. Wycofanie produktu (DELETE, oba węzły)

```
mutation {
  deleteProduct(sku: "WEB-001") {
    ok
    message
  }
}
```

Cel: wycofanie produktu ze sprzedaży (usunięcie logiczne).

Wynik: `ok = true`, komunikat `Produkt oznaczono jako nieaktywny atomowo w obu węzłach (PostgreSQL 2PC)`. Kolejne zapytanie `products` zwróciło 30 aktywnych produktów (bez WEB-001, z dodanym SSD-001).

10. Wyniki testów systemu rozproszonego

Czasy mierzono po stronie klienta, od wysłania zapytania do otrzymania odpowiedzi.

Wyniki liczbowe zależą od sprzętu, bieżącego obciążenia i konfiguracji (w Dockerze mogą być inne). Zestaw `Tests/` zapisuje wyniki własnego uruchomienia w plikach `report.html`, `report.json` i `measurements.csv`, więc na innym komputerze można je łatwo powtórzyć.

Pomiary z sekcji 10.1, 10.2 i 10.3 oraz test awarii Krakowa z zestawu `Tests/` w sekcji 10.4 pochodzą z raportu `/reports/tests`, który uruchomiono dwukrotnie: poleceniem python i w Dockerze (w kontenerze test_runner, bez testu awarii). Pozostałe pomiary awarii (sekcja 10.4), transakcji 2PC (sekcje 10.5 i 10.6) oraz dodatkowe pomiary równoległości i zapisu (sekcja 10.2) wykonano osobnymi skryptami na lokalnych instancjach PostgreSQL bez Dockera, więc ich bezwzględne czasy nie są porównywalne z czasami z raportu.

10.1. Test czasu odpowiedzi

Zestaw `Tests/` dodaje do bazy N produktów testowych `PERF-*` (100, 1 000 i 10 000) i dla każdego rozmiaru wykonuje jedno zapytanie o katalog (`products`). Test sprawdza, że zapytanie nie zwraca błędu i zawiera dokładnie oczekiwaną liczbę produktów `PERF-*`. Ten sam pomiar zestaw powtarza w teście skalowalności.

Wniosek: czas odpowiedzi rośnie liniowo z liczbą produktów w katalogu. Dla 100 produktów testowych wyniósł około 0,52–0,57 s, dla 1 000 około 4,35–4,46 s, a dla 10 000 około 42,2–45,8 s, czyli około 4 do 4,6 ms na produkt. Powtórzenia w teście skalowalności oraz oba uruchomienia (python i Docker) dały zgodne wyniki (różnice do około 9%). Wynika to z budowy kodu: każdy rekord i każde jego pole jest pobierane osobno, w osobnej sesji bazodanowej, zamiast jednym zapytaniem zbiorczym, dlatego dalsza optymalizacja odczytu produktów mogłaby poprawić wydajność systemu. Wcześniejsze pomiary osobnym skryptem na lokalnym PostgreSQL bez Dockera dały ten sam charakter wzrostu przy około dziesięć razy krótszych czasach, więc bezwzględne czasy zależą od środowiska. Wyniki obu uruchomień zestawiono w tabeli w sekcji 10.7.

Rysunek 1. Raport zestawu `Tests/` (uruchomienie poleceniem python): test czasu odpowiedzi.

6.1. Czas odpowiedzi

Cel	Pomiar odczytu katalogu dla rosnącej liczby produktów.
Metoda	Dodaj dane PERF-* i wykonaj jedno zapytanie katalogowe dla każdego rozmiaru.
Oczekiwany rezultat	Zapytanie nie zwraca błędu i zawiera dokładnie oczekiwaną liczbę produktów PERF-*.

WARIANT	WYNIKI	STATUS	WNIOSEK
100 produktów	**czas odpowiedzi (ms):** 554.423 **produkty testowe:** 100 **wszystkie produkty:** 131	PASS	API zwróciło komplet danych testowych.
1 000 produktów	**czas odpowiedzi (ms):** 4428.375 **produkty testowe:** 1000 **wszystkie produkty:** 1031	PASS	API zwróciło komplet danych testowych.
10 000 produktów	**czas odpowiedzi (ms):** 42216.411 **produkty testowe:** 10000 **wszystkie produkty:** 10031	PASS	API zwróciło komplet danych testowych.

Rysunek 2. Raport zestawu `Tests/` (uruchomienie w Dockerze): test czasu odpowiedzi.

6.1. Czas odpowiedzi

Cel	Pomiar odczytu katalogu dla rosnącej liczby produktów.
Metoda	Dodaj dane PERF-* i wykonaj jedno zapytanie katalogowe dla każdego rozmiaru.
Oczekiwany rezultat	Zapytanie nie zwraca błędu i zawiera dokładnie oczekiwaną liczbę produktów PERF-*.

WARIANT	WYNIKI	STATUS	WNIOSEK
100 produktów	**czas odpowiedzi (ms):** 520.884 **produkty testowe:** 100 **wszystkie produkty:** 131	PASS	API zwróciło komplet danych testowych.
1 000 produktów	**czas odpowiedzi (ms):** 4460.863 **produkty testowe:** 1000 **wszystkie produkty:** 1031	PASS	API zwróciło komplet danych testowych.
10 000 produktów	**czas odpowiedzi (ms):** 45776.507 **produkty testowe:** 10000 **wszystkie produkty:** 10031	PASS	API zwróciło komplet danych testowych.

Rysunek 3. Raport zestawu `Tests/` (uruchomienie poleceniem python): test skalowalności.

6.5. Skalowalność

Cel	Porównanie czasu odczytu dla rosnącego wolumenu danych.
Metoda	Dodaj 100, 1 000 i 10 000 produktów oraz zmierz odczyt katalogu.
Oczekiwany rezultat	Raport pokazuje czasy dla każdego rozmiaru bez błędów API.

WARIANT	WYNIKI	STATUS	WNIOSEK
100 produktów	**czas odpowiedzi (ms):** 566.524 **produkty testowe:** 100 **wszystkie produkty:** 131	PASS	API zwróciło komplet danych testowych.
1 000 produktów	**czas odpowiedzi (ms):** 4367.168 **produkty testowe:** 1000 **wszystkie produkty:** 1031	PASS	API zwróciło komplet danych testowych.
10 000 produktów	**czas odpowiedzi (ms):** 45455.11 **produkty testowe:** 10000 **wszystkie produkty:** 10031	PASS	API zwróciło komplet danych testowych.

Rysunek 4. Raport zestawu `Tests/` (uruchomienie w Dockerze): test skalowalności.

6.5. Skalowalność

Cel	Porównanie czasu odczytu dla rosnącego wolumenu danych.
Metoda	Dodaj 100, 1 000 i 10 000 produktów oraz zmierz odczyt katalogu.
Oczekiwany rezultat	Raport pokazuje czasy dla każdego rozmiaru bez błędów API.

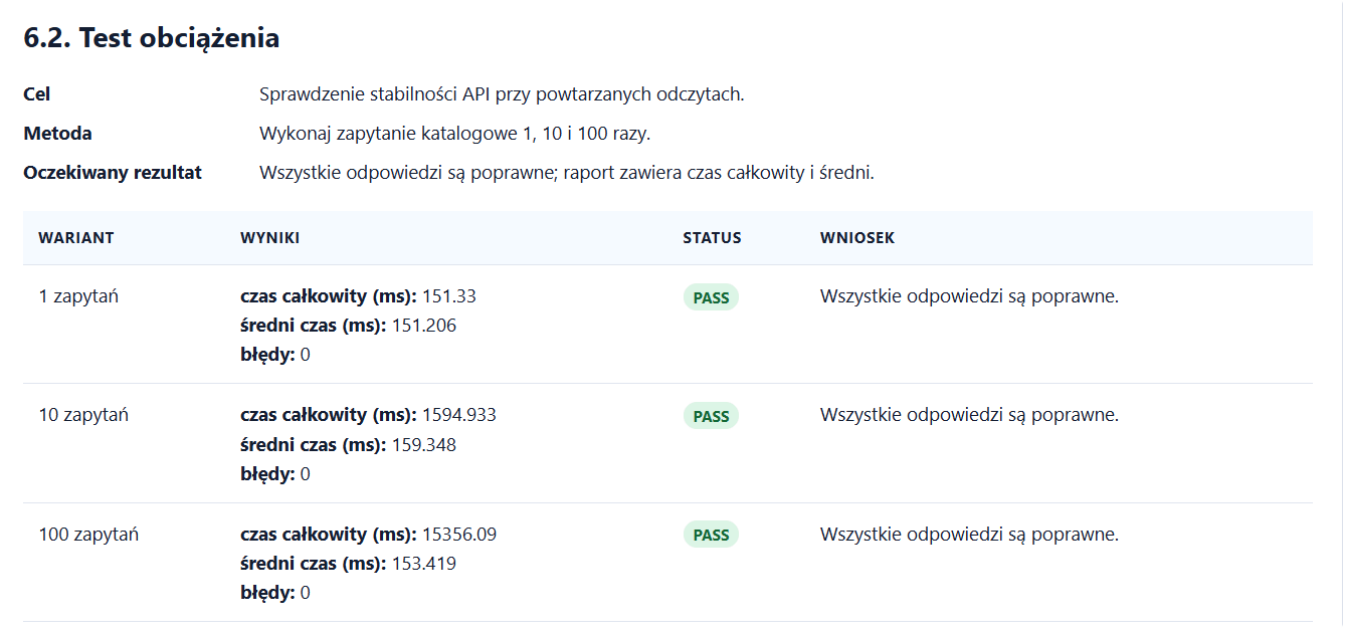
WARIANT	WYNIKI	STATUS	WNIOSEK
100 produktów	**czas odpowiedzi (ms):** 557.329 **produkty testowe:** 100 **wszystkie produkty:** 131	PASS	API zwróciło komplet danych testowych.
1 000 produktów	**czas odpowiedzi (ms):** 4346.417 **produkty testowe:** 1000 **wszystkie produkty:** 1031	PASS	API zwróciło komplet danych testowych.
10 000 produktów	**czas odpowiedzi (ms):** 42165.704 **produkty testowe:** 10000 **wszystkie produkty:** 10031	PASS	API zwróciło komplet danych testowych.

10.2. Test obciążenia

W teście wykonano 1, 10 i 100 kolejnych zapytań odczytujących katalog produktów (`products`). Zmierzono łączny czas wykonania oraz średni czas pojedynczego zapytania. Wszystkie zapytania zakończyły się powodzeniem; nie odnotowano błędów.

Wniosek: wszystkie 111 zapytań w każdym z dwóch uruchomień zakończyło się poprawnie. Łączny czas wykonania wzrósł wraz z liczbą zapytań: od około 148–151 ms dla jednego zapytania do 13 927–15 356 ms dla 100 zapytań. Średni czas pojedynczego zapytania utrzymywał się na zbliżonym poziomie (od 139,164 do 159,348 ms). Wyniki wskazują, że przy większej liczbie odczytów rośnie przede wszystkim łączny czas obsługi żądań, natomiast średni czas pojedynczego zapytania pozostaje stosunkowo stabilny. Wyniki obu uruchomień zestawiono w tabeli w sekcji 10.7.

Rysunek 5. Raport zestawu `Tests/` (uruchomienie poleceniem python): test obciążenia.



6.2. Test obciążenia

Cel Sprawdzenie stabilności API przy powtarzanych odczytach.

Metoda Wykonaj zapytanie katalogowe 1, 10 i 100 razy.

Oczekiwany rezultat Wszystkie odpowiedzi są poprawne; raport zawiera czas całkowity i średni.

WARIANT	WYNIKI	STATUS	WNIOSEK
1 zapytań	**czas całkowity (ms):** 151.33 **średni czas (ms):** 151.206 **błędy:** 0	PASS	Wszystkie odpowiedzi są poprawne.
10 zapytań	**czas całkowity (ms):** 1594.933 **średni czas (ms):** 159.348 **błędy:** 0	PASS	Wszystkie odpowiedzi są poprawne.
100 zapytań	**czas całkowity (ms):** 15356.09 **średni czas (ms):** 153.419 **błędy:** 0	PASS	Wszystkie odpowiedzi są poprawne.

Rysunek 6. Raport zestawu `Tests/` (uruchomienie w Dockerze): test obciążenia.

6.2. Test obciążenia

Cel Sprawdzenie stabilności API przy powtarzanych odczytach.

Metoda Wykonaj zapytanie katalogowe 1, 10 i 100 razy.

Oczekiwany rezultat Wszystkie odpowiedzi są poprawne; raport zawiera czas całkowity i średni.

WARIANT	WYNIKI	STATUS	WNIOSEK
1 zapytań	**czas całkowity (ms):** 148.42 **średni czas (ms):** 148.266 **błędy:** 0	PASS	Wszystkie odpowiedzi są poprawne.
10 zapytań	**czas całkowity (ms):** 1428.912 **średni czas (ms):** 142.79 **błędy:** 0	PASS	Wszystkie odpowiedzi są poprawne.
100 zapytań	**czas całkowity (ms):** 13926.992 **średni czas (ms):** 139.164 **błędy:** 0	PASS	Wszystkie odpowiedzi są poprawne.

Dodatkowo osobnym skryptem na lokalnym PostgreSQL (bez Dockera) zmierzono odczyt z obu węzłów, równoległość 10 wątków i zapis zamówień:

Porównano 1, 10 i 100 zapytań na danych z sekcji 8 (62 rekordy w zapytaniu `inventory`), wysyłanych kolejno oraz równolegle przez 10 wątków. Wszystkie zapytania zakończyły się powodzeniem.

Zapytanie	Liczba zapytań	Wątki	Czas całkowity	Średni czas zapytania	Maks.
odczyt `inventory` (2 węzły)	1	1	67 ms	66 ms	66 ms

Zapytanie	Liczba zapytań	Wątki	Czas całkowity	Średni czas zapytania	Maks.
odczyt `inventory` (2 węzły)	10	1	727 ms	73 ms	104 ms
odczyt `inventory` (2 węzły)	100	1	6 393 ms	64 ms	105 ms
odczyt `inventory` (2 węzły)	10	10	566 ms	296 ms	560 ms
odczyt `inventory` (2 węzły)	100	10	6 436 ms	617 ms	803 ms
zapis `createOrder`	1	1	8 ms	8 ms	8 ms
zapis `createOrder`	10	1	47 ms	5 ms	5 ms
zapis `createOrder`	100	1	516 ms	5 ms	26 ms
zapis `createOrder`	10	10	46 ms	25 ms	41 ms
zapis `createOrder`	100	10	462 ms	44 ms	58 ms

Wniosek: przy zapytaniach wysyłanych kolejno czas całkowity rośnie liniowo z ich liczbą (około 65 ms na odczyt i 5 ms na zapis). Równoległość 10 wątków nie skróciła czasu całkowitego odczytów (100 odczytów: 6 393 ms kolejno i 6 436 ms równolegle), a średni czas pojedynczego zapytania wzrósł z 64 do 617 ms. Sugeruje to, że pojedynczy proces API przetwarza zapytania odczytu głównie szeregowo. Zapisy są szybsze i przy 10 wątkach skróciły czas całkowity tylko nieznacznie (516 ms kolejno, 462 ms równolegle).

10.3. Test spójności danych

Test z raportu `Tests/` (rysunki 7 i 8) sprawdza replikację katalogu i lokalny stan magazynowy: dla SKU `CONS-BC532EF28A` (uruchomienie python) i `CONS-68E5C7D8C4` (Docker) w obu uruchomieniach wynik to Katowice – 777, Kraków – 0. Poniżej testy ręczne:

Test	Działanie	Wynik
T1	`createProduct` z SKU CON-001 (test ręczny)	Produkt natychmiast widoczny w obu bazach (cena 100 w Katowicach i w Krakowie). Zapis idzie w jednej transakcji 2PC w tym samym żądaniu, więc nie ma opóźnienia, a po zapisie w obu bazach nie zostają żadne transakcje przygotowane.
T2	`updateInventory` w Katowicach na 42 szt.	Katowice: 42, Kraków: 0. Stany magazynowe są lokalne, więc tak ma być.

Test	Działanie	Wynik
T3	Zmiana ceny CON-001 na 120 bezpośrednio w bazie Katowic (poza aplikacją)	Katowice: 120, Kraków: nadal 100. Brak automatycznej synchronizacji, a API (czytające katalog z Katowic) pokazuje 120.
T4	Zamówienie przekraczające stan magazynowy	Błąd, w tabeli `orders` nie został żaden rekord (transakcja wycofana).

Wniosek: test z raportu potwierdził, że katalog produktów jest dostępny w obu bazach, a stan magazynowy pozostaje lokalny dla oddziału. Wartość 777 w Katowicach i 0 w Krakowie pokazuje, że zmiana stanu w jednym oddziale nie powoduje automatycznej zmiany stanu w drugim. Testy ręczne pokazały ponadto, że dane są spójne, dopóki zmiany przechodzą przez aplikację, a zmiana katalogu wykonana poza nią powoduje rozbieżność między węzłami (T3).

Rysunek 7. Raport zestawu `Tests/` (uruchomienie poleceniem python): test spójności danych.

6.3. Spójność danych

Cel	Sprawdzenie replikacji katalogu i lokalności magazynu.
Metoda	Utwórz produkt, zmień stan Katowic i odczytaj oba oddziały.
Oczekiwany rezultat	Katalog jest w obu bazach, a zmieniony stan występuje wyłącznie w Katowicach.

WARIANT	WYNIKI	STATUS	WNIOSEK
replikacja katalogu i lokalny stan magazynowy	**SKU:** CONS-BC532EF28A **Katowice:** 777 **Kraków:** 0	PASS	Katalog został zreplikowany, a stan magazynowy pozostał lokalny.

Rysunek 8. Raport zestawu `Tests/` (uruchomienie w Dockerze): test spójności danych.

6.3. Spójność danych

Cel	Sprawdzenie replikacji katalogu i lokalności magazynu.
Metoda	Utwórz produkt, zmień stan Katowic i odczytaj oba oddziały.
Oczekiwany rezultat	Katalog jest w obu bazach, a zmieniony stan występuje wyłącznie w Katowicach.

WARIANT	WYNIKI	STATUS	WNIOSEK
replikacja katalogu i lokalny stan magazynowy	**SKU:** CONS-68E5C7D8C4 **Katowice:** 777 **Kraków:** 0	PASS	Katalog został zreplikowany, a stan magazynowy pozostał lokalny.

10.4. Test awarii

Węzeł Kraków wyłączono poleceniem `pg_ctlcluster 16 krk stop`, a następnie wykonano po kolei zapytania.

Operacja przy wyłączonym Krakowie	Wynik
`products` (katalog)	Działa (odczyt z Katowic).
`inventory(node:"KATOWICE")`	Działa.
`createOrder` w Katowicach	Działa.
`inventory(node:"KRAKOW")`, `createOrder` w Krakowie	Błąd `connection refused`.
`inventory` i `orders` bez `node` (oba węzły)	Błąd całego zapytania, bez częściowych wyników z działających Katowic.
`createProduct` (oba węzły)	Błąd `connection refused`. Produkt TPC-A nie został zapisany w żadnym węźle (0 rekordów w Katowicach, 0 transakcji przygotowanych).
`deleteProduct` (oba węzły)	Błąd. Produkt MON-001 nadal ma `active = true` w Katowicach.

Po ponownym uruchomieniu Krakowa (bez restartu API):

- `inventory` dla Krakowa, `orders` z obu węzłów i `createOrder` w Krakowie działają poprawnie, a dane Krakowa nie zostały utracone.
- Ponowne `createProduct` (TPC-A) kończy się powodzeniem (`Produkt zapisano atomowo w obu węzłach (PostgreSQL 2PC)`), a katalogi w obu węzłach są identyczne.

Po wyłączeniu Katowic (`pg_ctlcluster 16 kat stop`):

Operacja przy wyłączonych Katowicach	Wynik
`products` (katalog)	Błąd, bo katalog jest czytany z Katowic.
`inventory(node:"KRAKOW")`	Działa.

Operacja przy wyłączonych Katowicach	Wynik
`createOrder` w Krakowie	Działa.
`inventory` bez `node` (oba węzły)	Błąd całego zapytania.
`createProduct`, `deleteProduct`	Błąd. W Krakowie brak zmian (0 rekordów TPC-K, KEY-001 nadal aktywny) i brak transakcji przygotowanych.

Po restarcie Katowic ponowne `createProduct` (TPC-K) kończy się powodzeniem.

Test awarii z zestawu `Tests/` zatrzymuje PostgreSQL Kraków, wykonuje odczyty, uruchamia węzeł ponownie i sprawdza odzyskanie:

Test awarii zatrzymuje kontener Docker Compose i wymaga dostępu do Docker Daemon, dlatego kontener `test_runner` (profil `tests`) domyślnie go pomija. Wykonano go wyłącznie z hosta poleceniem `python -m Tests.run_tests` (Rysunek 9); w uruchomieniu w Dockerze nie był wykonywany.

Wariant	Wynik	Status
Podczas awarii Krakowa	Katalog dostępny: tak. Katowice dostępne: tak. Oba węzły dostępne: nie. Błąd: `could not translate host name "postgres_krakow" to address: Name or service not known`.	PASS
Po restarcie Krakowa	Katalog dostępny: tak. Katowice dostępne: tak. Oba węzły dostępne: tak. Błędy: brak.	PASS

Wniosek: Katowice są dostępne mimo awarii Krakowa, a po jego ponownym uruchomieniu oba węzły wracają do działania bez ręcznej naprawy. Wynik zgadza się z testami ręcznymi opisanymi powyżej.

Rysunek 9. Raport zestawu `Tests/` (uruchomienie poleceniem python): test awarii.

6.4. Test awarii

Cel	Sprawdzenie dostępności Katowic przy awarii Krakowa.
Metoda	Zatrzymaj PostgreSQL Kraków, wykonaj odczyty, uruchom go ponownie i sprawdź odzyskanie.
Oczekiwany rezultat	Katowice pozostają dostępne podczas awarii, a oba węzły działają po restarcie.

WARIANT	WYNIKI	STATUS	WNIOSEK
podczas awarii Krakowa	**katalog dostępny:** True **Katowice dostępne:** True **oba węzły dostępne:** False **błędy obu węzłów:** ['could not translate host name "postgres_krakow" to address: Name or service not known\n']	PASS	Katowice są dostępne mimo awarii drugiego węzła.
po restarcie Krakowa	**katalog dostępny:** True **Katowice dostępne:** True **oba węzły dostępne:** True **błędy obu węzłów:** []	PASS	Katowice są dostępne mimo awarii drugiego węzła.

10.5. Test awarii w trakcie transakcji dwufazowej (2PC)

Awarie wywoływano wprost w kodzie koordynatora (`backend/catalog_transaction.py`). W scenariuszu B wyłączono Kraków w chwili między zapisem decyzji commit a `COMMIT PREPARED` dla Krakowa, a w scenariuszu C zakończono proces API tuż po przygotowaniu obu węzłów, przed zapisem decyzji. Stan sprawdzano zapytaniami do obu baz (liczba rekordów produktu i liczba transakcji w `pg_prepared_xacts`).

Scenariusz	Stan po awarii	Po odzyskaniu
A. Kraków wyłączony przed zapisem (przez API)	Błąd `connection refused`. Katowice: 0 rekordów TPC-A, MON-001 nadal aktywny, 0 transakcji przygotowanych.	Ponowne `createProduct` udane.
B. Kraków pada po zapisaniu decyzji commit	Katowice: produkt widoczny (1 rekord), 0 przygotowanych. Po restarcie Krakowa: produkt niewidoczny (0 rekordów) i 1 transakcja w stanie prepared (przejściowa niespójność).	Następna zmiana katalogu przez API uruchomiła odzyskiwanie: Kraków ma produkt (1 rekord), w obu węzłach 0 przygotowanych transakcji.

Scenariusz	Stan po awarii	Po odzyskaniu
C. Proces API ginie po przygotowaniu obu węzłów, przed decyzją	Oba węzły: 0 widocznych rekordów produktu i po 1 transakcji w stanie prepared.	Po restarcie API (odzyskiwanie przy starcie) transakcje zostały wycofane: 0 przygotowanych, produkt nie istnieje.
D. Duplikat SKU	Błąd `Produkt o SKU TPC-A już istnieje`, 0 transakcji przygotowanych.	Nie dotyczy.

Wniosek: zmiana katalogu jest wycofywana albo dokańczana, więc nie zostaje trwale zapisana w jednym węźle. Jedyne okno niespójności to czas od zatwierdzenia w jednym węźle do odzyskania drugiego (scenariusz B). Odzyskiwanie nie działa w tle: przygotowana transakcja czeka na restart API albo następną zmianę katalogu. Kod nie usuwa starych wpisów z tabeli decyzji.

10.6. Koszt transakcji 2PC

Po 20 wywołań każdej mutacji (ta sama konfiguracja, dane demonstracyjne):

Operacja	Średnio	Mediana	Maks.
`createProduct` (2PC)	133 ms	119 ms	187 ms
`deleteProduct` (2PC)	105 ms	103 ms	144 ms

Dla porównania wcześniejsza wersja bez 2PC (sprawdzenie obu baz przed zapisem) potrzebowała około 40 do 50 ms na pojedyncze wywołanie (pojedyncze pomiary). Transakcja dwufazowa jest więc około 2 do 3 razy wolniejsza, bo wymaga dodatkowych połączeń, `PREPARE TRANSACTION` i zapisu decyzji, a w zamian daje atomowość zapisu katalogu.

10.7. Podsumowanie testów i skalowalność

Test	Wariant	Wynik	Wniosek
Czas odpowiedzi	100 / 1 000 / 10 000 produktów testowych	Python: 554,423 / 4 428,375 / 42 216,411 ms. Docker: 520,884 / 4 460,863 / 45 776,507 ms	Wzrost liniowy (około 4 do 4,6 ms na produkt), API zwróciło komplet danych.
Skalowalność	powtórzenie: 100 / 1 000 / 10 000 produktów testowych	Python: 566,524 / 4 367,168 / 45 455,11 ms. Docker: 557,329 / 4 346,417 / 42 165,704 ms	Wyniki zgodne z pierwszym pomiarem (różnice do około 9%).
Obciążenie	1 / 10 / 100 odczytów katalogu	Python: 151,33 / 1 594,933 / 15 356,09 ms. Docker: 148,42 / 1 428,912 / 13 926,992 ms (łącznie)	Wzrost liniowy, stały średni czas (139 do 159 ms), 0 błędów.
Obciążenie	100 odczytów, 10 wątków (pomiar lokalny)	6,4 s (kolejno 6,4 s)	Równoległość nie poprawia czasu całkowitego.
Spójność	replikacja katalogu i lokalny stan SKU `CONS-BC532EF28A` (python) i `CONS-68E5C7D8C4` (Docker)	Katowice: 777, Kraków: 0	Katalog został zreplikowany, stan magazynowy pozostał lokalny.
Spójność	zapis katalogu przez aplikację (2PC)	natychmiast w obu węzłach	Poprawnie.
Spójność	zmiana poza aplikacją	rozbieżność	Brak replikacji na poziomie bazy.
Awaria	Kraków wyłączony (test z raportu)	katalog i Katowice dostępne, oba węzły niedostępne; po restarcie wszystko działa	Izolacja operacji lokalnych działa, odzyskanie bez ręcznej naprawy.

Test	Wariant	Wynik	Wniosek
Awaria	Katowice wyłączone	Kraków obsługuje lokalne operacje, `products` zawodzi	Katalog zależy od Katowic.
Awaria	zmiana katalogu przy niedostępnym węźle	błąd, brak zmian w katalogach	2PC chroni przed zapisem w jednym węźle.
Awaria	awaria w trakcie 2PC	wycofanie albo dokończenie po odzyskaniu węzła	Atomowość z krótkim oknem niespójności.
Koszt	`createProduct` / `deleteProduct` (2PC)	133 ms / 105 ms	Około 2 do 3 razy wolniej niż bez 2PC.

Porównanie podsumowań raportu `Tests/` z obu uruchomień. Wszystkie wykonane testy zakończyły się wynikiem PASS, a czas sekcji obejmuje też przygotowanie danych testowych:

Sekcja raportu	Scenariusz	Python: status	Docker: status	Python: czas sekcji	Docker: czas sekcji
6.1	Czas odpowiedzi	PASS	PASS	53 202,672 ms	52 680,607 ms
6.2	Test obciążenia	PASS	PASS	17 103,611 ms	15 480,928 ms
6.3	Spójność danych	PASS	PASS	787,551 ms	750,005 ms
6.4	Test awarii	PASS	nie uruchamiano (pomijany w kontenerze `test_runner`)	14 467,279 ms	–
6.5	Skalowalność	PASS	PASS	56 167,160 ms	53 202,028 ms

11. Wnioski końcowe

- **Czy rozproszenie danych działa poprawnie?** Tak. Zamówienia i stany magazynowe są zapisywane lokalnie w węźle oddziału, a zapytania `inventory` i `orders` bez parametru `node` poprawnie łączą dane z obu węzłów.

- **Czy dane pozostają spójne?** W normalnej pracy tak. Zmiana katalogu jest zapisywana w transakcji dwufazowej: albo trafia do obu węzłów, albo jest wycofana, a awaria po podjęciu decyzji commit jest dokańczana po odzyskaniu węzła (do tego czasu zmiana może być widoczna tylko w jednym węźle). Spójność nie jest gwarantowana przy zmianach wykonanych poza aplikacją.
- **Jak zmienia się czas odpowiedzi?** W teście odczytu katalogu czas wzrósł liniowo z około 0,52–0,57 s dla 131 produktów do około 42–46 s dla 10 031 produktów, czyli około 4 do 4,6 ms na produkt (sekcja 10.1). W teście obciążenia średni czas pojedynczego odczytu utrzymywał się na poziomie około 139–159 ms (sekcja 10.2). Główną przyczyną wzrostu jest pobieranie każdego rekordu i pola osobno. Zmiana katalogu w 2PC trwała średnio 105–133 ms, czyli około 2 do 3 razy dłużej niż we wcześniejszym pomiarze bez 2PC (sekcja 10.6).
- **Co dzieje się przy awarii jednego węzła?** Operacje lokalne w drugim węźle działają dalej (przy awarii Katowic bez odczytu katalogu). Zapytania obejmujące oba węzły zwracają błąd zamiast częściowych wyników, a zmiany katalogu są odrzucane bez zapisu w jednym węźle. Awaria w trakcie 2PC kończy się wycofaniem albo dokończeniem zmiany po odzyskaniu węzła lub restarcie API.

Zalety: prosta architektura, niezależne działanie oddziałów, izolacja awarii dla danych lokalnych, atomowy zapis katalogu w obu węzłach dzięki transakcji dwufazowej, łatwe uruchomienie jednym poleceniem, powtarzalna demonstracja dzięki idempotentnemu seedowi i automatycznemu raportowi testów (HTML, JSON, CSV) oraz flaga `active`, która chroni przed zamówieniem wycofanych produktów.

Ograniczenia: transakcja dwufazowa jest blokująca (przygotowana transakcja czeka na odzyskanie węzła, a odzyskiwanie działa przy restarcie API i przed kolejną zmianą katalogu, a nie w tle), koordynator zapisuje decyzję tylko w Katowicach (przy ich awarii zmiany katalogu są odrzucane), 2PC działa tylko dla PostgreSQL, brak replikacji zmian wykonanych bezpośrednio w bazie, brak logowania i autoryzacji, odczyt katalogu zależny od Katowic, brak częściowych wyników przy awarii, identyfikatory niepowtarzalne tylko w obrębie węzła oraz wydajność ograniczona sposobem pobierania danych.

Możliwe usprawnienia: okresowe odzyskiwanie przygotowanych transakcji w tle, czyszczenie starych wpisów z tabeli decyzji, replikacja logiczna PostgreSQL (albo kolejka zdarzeń) zamiast 2PC dla tabeli `products`, zwracanie częściowych wyników z działających węzłów, pobieranie danych jednym zapytaniem zbiorczym zamiast osobno dla każdego pola oraz uwierzytelnianie użytkowników.